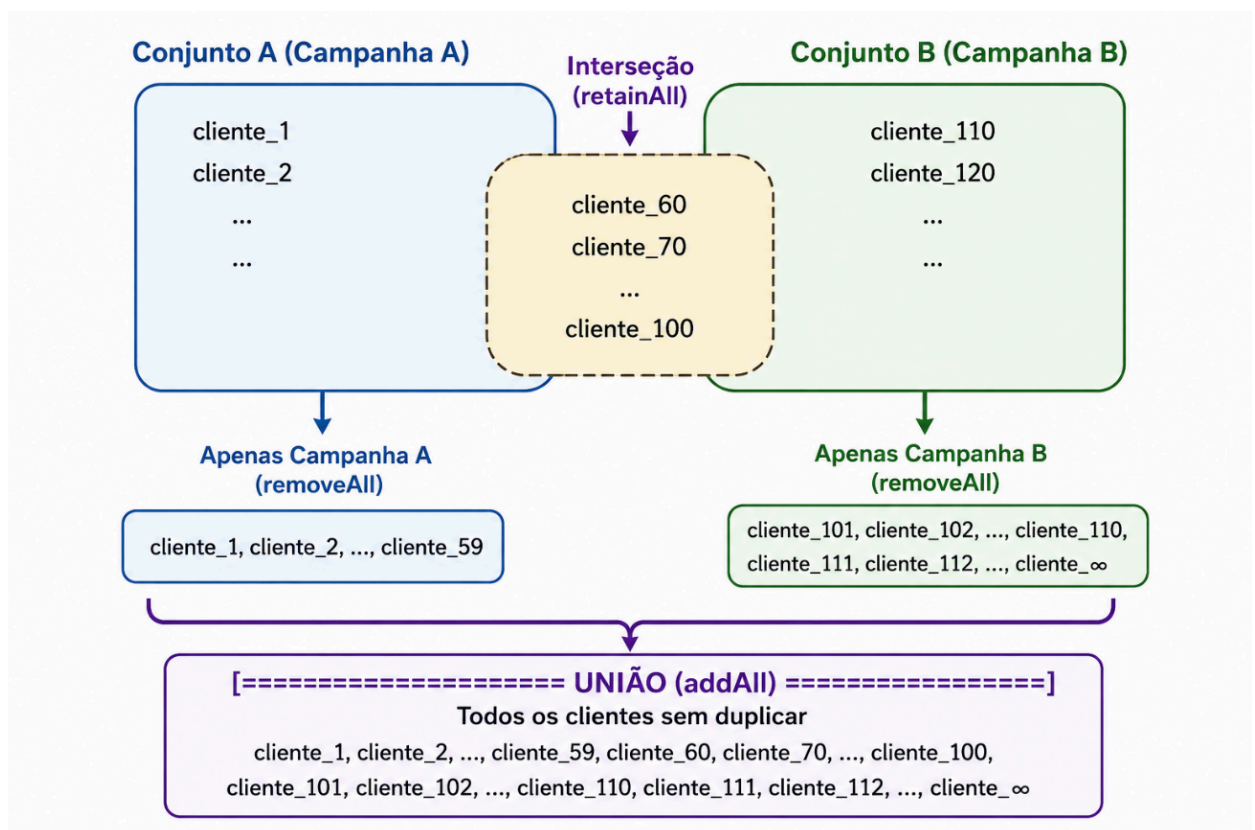


HASHSET ESTRUTURA DE DADOS I

O HashSet é uma coleção do Java baseada em uma tabela hash que armazena elementos únicos e não ordenados, destacando-se no mercado por sua altíssima eficiência de busca, inserção e remoção em tempo constante ($O(1)$). Em vez de percorrer uma lista item por item, ele utiliza o método `hashCode()` do objeto para calcular instantaneamente sua posição, rejeitando de forma automática qualquer tentativa de duplicata e tornando-se a estrutura ideal para operações de alta performance, como filtragem de dados e lógica de conjuntos.

Diferente do HashMap, que mapeia pares de chave e valor para associar informações, o HashSet utiliza apenas a chave (o próprio objeto inserido) para verificar a unicidade dos elementos. Dessa forma, ele otimiza o uso da memória ao focar exclusivamente na existência do elemento: ao inserir um dado, o HashSet calcula seu `hashCode()` e usa essa chave para garantir que o objeto seja armazenado em uma posição única.

Exemplo de operações



Crie uma aplicação Java e insira as duas funções estáticas para simular a geração clientes de duas campanhas.

HASHSET ESTRUTURA DE DADOS I

Considere

```
private static ArrayList<String> gerarEmailsCampanhaA() {
    ArrayList<String> emails = new ArrayList<>();
    for (int i = 1; i <= 100; i++) {
        emails.add("cliente_" + i + "@email.com");
    }
    return emails;
}

private static ArrayList<String> gerarEmailsCampanhaB() {
    ArrayList<String> emails = new ArrayList<>();
    for (int i = 70; i < 160; i++) {
        emails.add("cliente_" + i + "@email.com");
    }
    return emails;
}

// main

ArrayList<String> listaCampanhaA = gerarEmailsCampanhaA();
List<String> listaCampanhaB = gerarEmailsCampanhaB();

System.out.println("=== SISTEMA DE SEGMENTAÇÃO DE CAMPANHAS ===");
System.out.println("Total de e-mails na Campanha A: " +
listaCampanhaA.size());
System.out.println("Total de e-mails na Campanha B: " +
listaCampanhaB.size());

System.out.println("-----
\n");
    for(String email: listaCampanhaA)
        System.out.println("Campanha A: " + email);

System.out.println("-----
\n");
    for(String email: listaCampanhaB)
        System.out.println("Campanha B: " + email);
```

HASHSET ESTRUTURA DE DADOS I

1.  Apresente o quantitativo de elementos em cada uma das listas. Caso você precise encontrar apenas os clientes que se encontram apenas na campanha A ou na campanha B, explique a lógica de código a partir do ArrayList

=== SISTEMA DE SEGMENTAÇÃO DE CAMPANHAS ===

Total de e-mails na Campanha A: 100

Total de e-mails na Campanha B: 90

Percorre os emails de uma lista para comparar com emails da outra lista para ver se existe na outra lista, mas tem uma complexidade de $O(n^2)$, criando uma nova lista.

Convertendo o ArrayList para HashSet e verificando únicos em cada campanha

```
HashSet<String> hashSetCampanhaA = new
HashSet<>(listaCampanhaA);
HashSet<String> hashSetCampanhaB = new
HashSet<>(listaCampanhaB);
//Removendo clientes da campanha B que estão na campanha A /
HashSet<String> clientesUnicosCampanhaA = new
HashSet<>(hashSetCampanhaA);
clientesUnicosCampanhaA.removeAll(hashSetCampanhaB);
// Removendo clientes da campanha A que estão na campanha B
HashSet<String> clientesUnicosCampanhaB = new
HashSet<>(hashSetCampanhaB);
clientesUnicosCampanhaB.removeAll(hashSetCampanhaA);
System.out.println("\n=== CLIENTES ÚNICOS DE CADA CAMPANHA
===");
System.out.println("Clientes únicos da Campanha A: " +
clientesUnicosCampanhaA.size());
for(String email: clientesUnicosCampanhaA)
System.out.println("Clientes únicos da Campanha A: " +
email);
System.out.println("Clientes únicos da Campanha B: " +
clientesUnicosCampanhaB.size());
for(String email: clientesUnicosCampanhaB)
System.out.println("Clientes únicos da Campanha B: " +
email);
```

HASHSET ESTRUTURA DE DADOS I

2.  Quantos clientes são únicos em cada campanha?

=== CLIENTES ÚNICOS DE CADA CAMPANHA ===

Clientes únicos da Campanha A: 69

Clientes únicos da Campanha B: 59

3.  Considerando a avaliação acima, utilize a função `retainAll` para obter os usuários duplicados, para isso utilize a mesma lógica para verificar os exclusivos de cada campanha: Crie um novo `HashSet` a partir da campanha A e aplique o `retainAll` passando o `hashSet` da campanha B, assim o mesmo removerá tudo que NÃO estiver contido no `hashSetCampanhaB`. Apresente o código.

```
HashSet<String> duplicados = new HashSet<>(listaCampanhaA);  
  
duplicados.retainAll(hashSetCampanhaB);
```

4.  Quantos registros são duplicados nos dois conjuntos?

=== CLIENTES DUPLICADOS ===

Clientes duplicados: 31

=== CLIENTES UNICOS ===

Clientes unicos: 159